

CAMADA DE SERVIÇOS / API (TESTES DE INTEGRAÇÃO E CONTRATO)

Frameworks de Automação de Testes: WireMock & Mockoon com IA

Simulação e interceptação de requisições de rede na prática

Diferencial: Diferente de testes unitários (como JUnit e Mockito), estas ferramentas sobem servidores HTTP locais que interceptam e simulam requisições de rede na prática.



Kayke Santos
Apresentador

O Problema das Dependências de APIs e Por Que Usar Mocks

OS GARGALOS DAS DEPENDÊNCIAS REALISTAS

Instabilidade no CI/CD

Falhas intermitentes ocorrem quando a API de terceiros fica indisponível.

Custos Excessivos

APIs pagas cobram por volume de chamadas durante baterias de testes.

Dificuldade de Testar Erros

É complexo forçar erros HTTP 500 ou timeouts reais em produção.

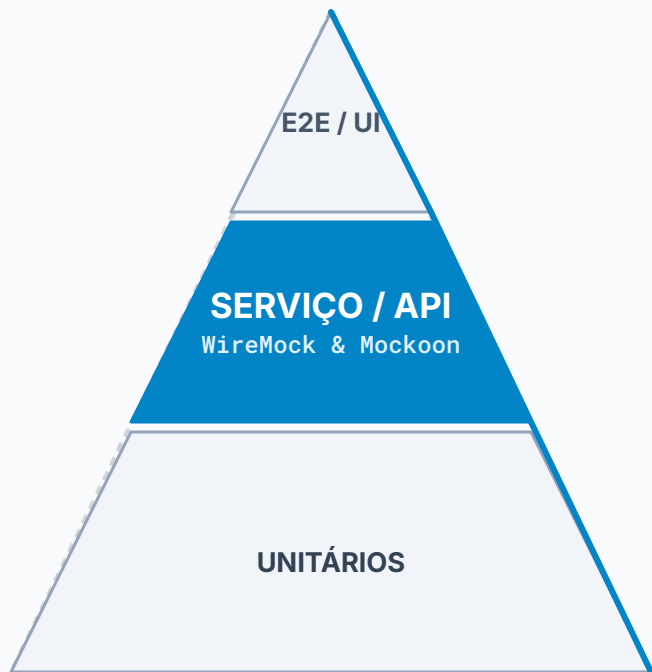
A SOLUÇÃO DEFINITIVA

API Mocking

Simulação controlada e previsível do comportamento das dependências.

- ✓ **Isolamento Total**
Independência de serviços de terceiros e rede.
- ✓ **Custo Zero em Testes**
Elimina a tarifação por requisições em staging.
- ✓ **Previsibilidade Absoluta**
Respostas determinísticas para cenários de sucesso e erro.
- ✓ **Alta Velocidade**
Desenvolvimento paralelo contínuo para Frontend e Backend.

Posicionamento na Pirâmide de Automação



03. TESTES DE INTERFACE / E2E

Testam a aplicação através da interface do usuário, avaliando fluxos completos.

02. TESTES DE SERVIÇO / API

Testam a comunicação entre componentes e serviços através de HTTP.

★ WireMock e Mockoon ficam principalmente nesta camada

01. TESTES UNITÁRIOS

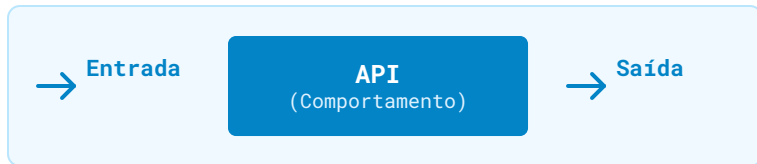
Testam unidades/componentes isolados da aplicação. (Ex: JUnit, Mockito)

Conclusão

WireMock e Mockoon atuam principalmente na camada de Serviço/API, simulando serviços HTTP para permitir testes isolados e previsíveis.

Por que Caixa-Preta e não Caixa-Branca?

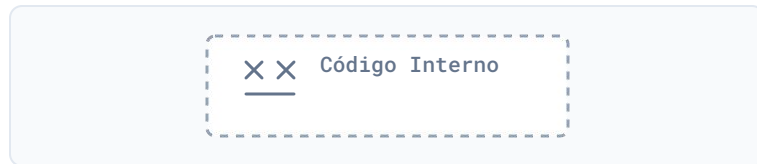
CAIXA-PRETA ✓



- Foca no comportamento externo
- Não precisa conhecer o código interno

WireMock / Mockoon

CAIXA-BRANCA



- Foca na implementação interna
- Analisa código e lógica detalhada

Mais comum em testes unitários (JUnit, Mockito)

Requisição HTTP



API Mockada



Resposta HTTP

WireMock e Mockoon testam principalmente o comportamento das APIs através de requisições e respostas HTTP, sem depender da implementação interna.

WireMock é o padrão da indústria para testes automatizados via stubs e execução headless.

ECOSSISTEMA CI/CD **Padrão da Indústria**

Ferramenta líder para simulação de APIs em pipelines de CI/CD contínuos.

Não apenas em java, mas também em javascript ...

REGRAS & RESPOSTAS **Conceito de Stubs**

Um stub basicamente define uma regra: quando chegar determinada requisição, devemos devolver determinada resposta.

GET /api/users, podemos configurar o WireMock para retornar um status 200 junto com um JSON

EXECUÇÃO REMOTA **Modo Headless**

Operação nativa via terminal e código, sem interface gráfica, ideal para automação contínua e execução em servidores remotos.

WireMock simulações

TIMEOUT

Delays de Latência

Simulação de respostas lentas (ex: 3 segundos) para validar timeouts e regras de retry da aplicação.

FAULT_INJECTION

Falhas TCP (Fault Injection)

Interrupção abrupta de conexões (`CONNECTION_RESET_BY_PEER`), respostas com 0 bytes ou corrupção de payload.

STATEFUL_SCENARIO

Stateful Mocks (Scenarios)

Máquinas de estado que simulam comportamentos complexos, como retornar **erro 500** na primeira tentativa e **sucesso 200** na segunda.

Mockoon acelera a prototipagem com uma interface visual intuitiva e geração de APIs via IA.

VISUAL UI

Interface Gráfica Intuitiva

Prototipagem rápida e visual, sem necessidade de escrever código ou scripts complexos.

INTELIGÊNCIA ARTIFICIAL

Assistente Nativo de IA

Geração automática de endpoints, rotas e payloads JSON completos a partir de descrições em linguagem natural.

PADRÕES & INTEGRAÇÃO

Compatibilidade OpenAPI/Swagger

Importação instantânea de contratos de API e uso conjunto com WireMock.

PRODUTIVIDADE LOCAL

Foco na Agilidade

Ideal para desenvolvedores Frontend e QA rodarem mocks localmente.

Mockoon gera respostas dinâmicas e realistas a cada requisição utilizando Faker.js e Handlebars.

```
GET /api/users/{{faker 'string.uuid'}}  
{ "name": "{{faker 'person.fullName'}}", "email": "{{faker 'internet.email'}}" }
```

- **Respostas Dinâmicas:** Diferente de mocks estáticos, o Mockoon altera os dados a cada chamada recebida.
- **Integração com Handlebars:** Uso de templates flexíveis para capturar dados da requisição e refleti-los na resposta.
- **Motor Faker.js Embutido:** Geração instantânea de dados realistas como nomes, e-mails, empresas e UUIDs.
- **Prototipagem Ágil:** Permite que equipes de frontend testem interfaces com dados variados antes do backend existir.

DEMONSTRAÇÃO PRÁTICA

**Vamos direto para o teste
prático**

WireMock domina a automação CI/CD, enquanto Mockoon prioriza a velocidade de prototipagem visual.

WireMock

- **Interface:** Headless / Código.
- **Curva de Aprendizado:** Exige conhecimento técnico.
- **Recursos de IA:** Utiliza IA via integrações externas.
- **Resiliência e Falhas:** Oferece simulação avançada de socket/TCP.
- **Foco Principal:** Pipelines CI/CD.

Mockoon

- **Interface:** Interface Gráfica rica.
- **Curva de Aprendizado:** Amigável e imediato.
- **Recursos de IA:** Possui assistente nativo de IA.
- **Resiliência e Falhas:** Foca em falhas HTTP padrão.
- **Foco Principal:** Desenvolvimento local e Frontend.

Líderes de tecnologia adotam WireMock para automação massiva e Mockoon para agilidade de desenvolvimento.

WireMock

[CASOS DE SUCESSO]

Utilizado por gigantes como **Netflix**, **Spotify**, **Uber** e **Nubank** para executar milhares de testes de integração automatizados diariamente.

[DESVANTAGEM]

Configuração inicial mais complexa e verbosa.

Mockoon

[CASOS DE SUCESSO]

Adotado por empresas como **Adobe** e **Siemens** para acelerar o desenvolvimento Frontend paralelo.

[DESVANTAGEM]

Menos adequado para simulação de falhas de rede em baixo nível no CI/CD.

WireMock e Mockoon são ferramentas complementares

Quando adotar WireMock

- **Microsserviços Backend:** Ideal para ecossistemas complexos e integrados.
- **Pipelines Automatizados:** Integração contínua perfeita via Docker, Jenkins e GitHub Actions.
- **Resiliência TCP:** Testes rigorosos de falha e comportamento de rede.

Quando adotar Mockoon

- **Prototipagem em Minutos:** Agilidade extrema com auxílio de Inteligência Artificial.
- **Descomplicação para Frontend:** Interface intuitiva para acelerar o desenvolvimento do client-side.
- **Testes Manuais:** Excelente para análises exploratórias e ad-hoc.

CONCLUSÃO E RECOMENDAÇÃO

"Não há competição direta entre as ferramentas."

Utilize o **Mockoon** na fase de design e desenvolvimento local para rápida iteração, e o **WireMock** para a validação automatizada e contínua nos pipelines de produção.